

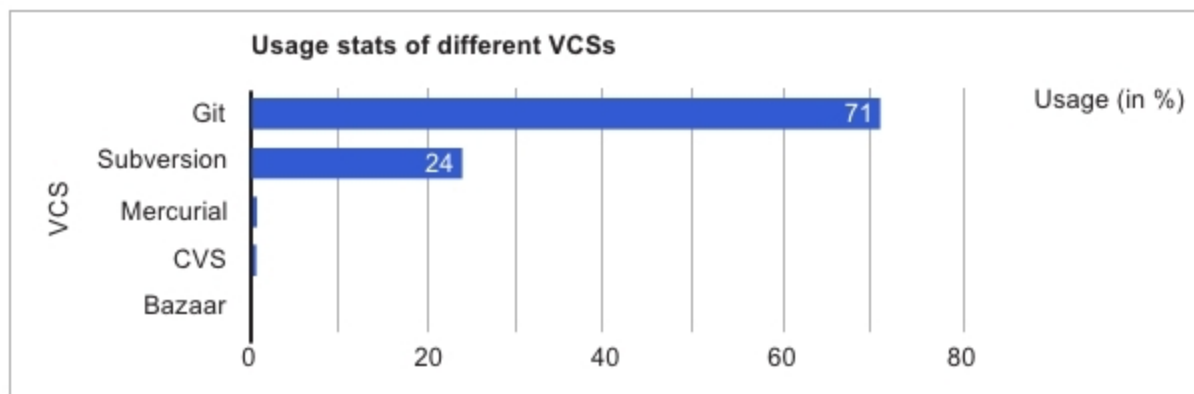


Figure 3: The popularity of different version control systems

```
(base) ekta@git-demo$ ls -l .git
total 32
drwxr-xr-x 2 ekta ekta 4096 May 7 19:05 branches
-rw-r--r-- 1 ekta ekta 92 May 7 19:05 config
-rw-r--r-- 1 ekta ekta 73 May 7 19:05 description
-rw-r--r-- 1 ekta ekta 23 May 7 19:05 HEAD
drwxr-xr-x 2 ekta ekta 4096 May 7 19:05 hooks
drwxr-xr-x 2 ekta ekta 4096 May 7 19:05 info
drwxr-xr-x 4 ekta ekta 4096 May 7 19:05 objects
drwxr-xr-x 4 ekta ekta 4096 May 7 19:05 refs
```

Figure 4: Directories/files inside a .git repo

A peek into a .git folder

Every Git repository contains a `.git` directory. This is where all the file's contents, commits and configuration reside, as shown in Figure 4.

The `branches` folder has been deprecated; hence, it is no longer in use. The `config` file contains project specific configuration options. The `description` is supposed to contain the repository name as set by the user but, by default, has the value `unnamed repository`. The `HEAD` points to the current branch checked out, which by default is a `master` in a newly created or cloned Git repository, and the `hooks` directory contains client or server side hooks scripts. The `info` directory contains a global 'exclude' file to keep track of patterns to ignore, mentioned in `.gitignore file.Objects`. And `refs` form the core part of Git, which is explained more in detail later.

The Git object model

At its core, Git is simply a key-value data store. Every bit of content that is added to Git has a unique key that can be later used to retrieve the contents. There are primarily two data structures that are used in Git — a mutable index that stores information about the working directory and an

immutable object database. Each object in the object database is identified by a unique SHA-1 hash. There are three parameters related to an object — size, content and type. 'Size' specifies the volume of content and 'type' specifies the type of content in the object. There are four types of objects.

- **Blob:** A binary large object (blob) is the content of the file. A new blob is created for every version of the file. This snapshot approach is different from the delta approach used in many VCSs, as discussed earlier.
- **Tree object:** This is a reference to blobs and other trees, and is an equivalent of a directory. The tree object contains one line per file or sub-directory, and it has information like the object type, object hash, file name, file permissions, etc.
- **Commit object:** This links all the tree objects together. It is a reference to the tree as well as all the parent commits, and contains metadata of all the changes introduced in the repository, like the author, committer, commit data, comment, etc.
- **Tag object:** This assigns a human readable name to a specific object, usually a commit object. It contains the object name, object type, tag name, tagger name and tag message. As stated, Git gives a unique

identifier (key) for the contents of each object, which is calculated by the SHA-1 hash. The identifier is 40 characters long, with the first two characters being the directory name and the remaining 38 characters the file/object name.

All these objects are stored in the `.git/objects` directory.

Git refs

It is necessary to know the hashes of Git objects in order to manipulate objects. It would be easier if simple names are used, instead of raw hashes. Git has references inside `.git/refs` to save the day. A *ref* is simply a text file somewhere in `.git/refs` containing a hash of a commit object. Its structure is shown in Figure 5.

```
(base) ekta@git-demo$ find .git/refs/
.git/refs/
.git/refs/remotes
.git/refs/remotes/origin
.git/refs/remotes/origin/master
.git/refs/heads
.git/refs/heads/master
.git/refs/heads/feature
.git/refs/tags
.git/refs/tags/v1.0
```

Figure 5: Structure of a ref in Git

- The `heads` directory defines the branches in the local repository, one file per branch, containing a SHA-1 commit hash of the tip of the branch. To change the tip of the branch, simply edit the `/heads/<branch-name>` file with the commit hash. Git also has the command `git update-ref` to achieve the same.
- The `remotes` directory contains the list of all remote repositories created using `git remote`. Each item in the list has a file with a branch name containing a SHA-1 commit hash of the tip of the branch, just like local branches.
- The tag *ref* created by a user points to a commit object unlike the branch *ref* that points to a tree object. The `tags` directory contains tag files, each containing the SHA-1 hash of the commit object it points to.

The Git workflow

Now that we have looked at the basic structure and characteristics of Git, let's have a brief look at its overall workflow. Being a distributed VCS,